

Thin Airfoil Solver

Poorvansh Jain

Section 1:

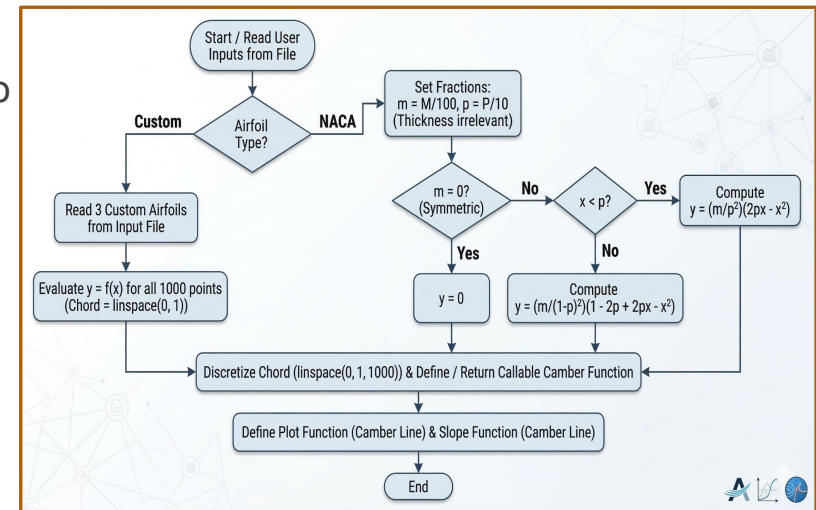
Algorithm

DESCRIBE THE ALGORITHM FOR EACH FUNCTION IN A STEPWISE MANNER. INCLUDE EQUATIONS WHEREVER RELEVANT.

EXAMPLES OF ALGORITHM : <https://www.programiz.com/dsa/algorithm#quadratic>

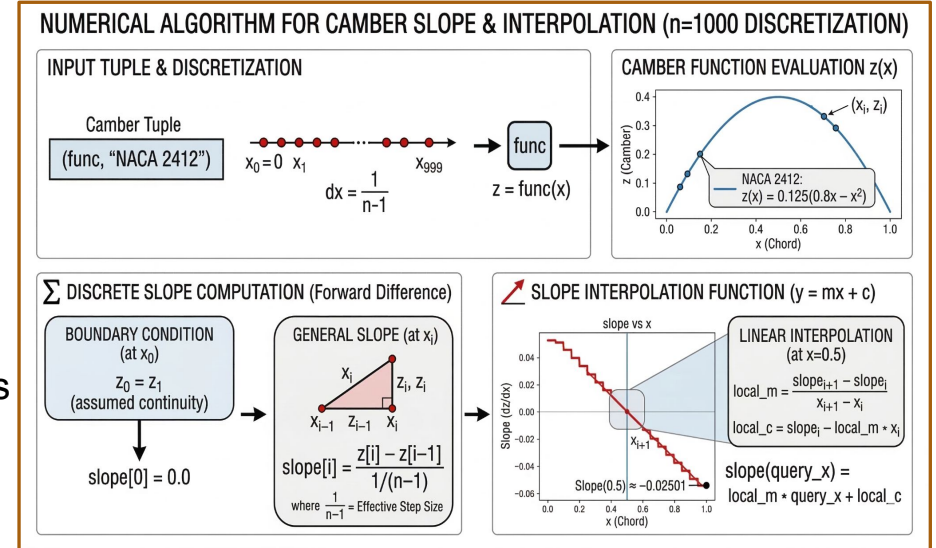
Algorithm for plotting camber line

1. Accept the user inputs through the input file
2. If NACA:
 - a. M (first digit) and P (second digit)
 - b. Thickness (last two digits) are irrelevant cause we will approximate it to a thin airfoil
 - c. Convert to fractions: $m = M/100$, $p = P/10$
 - d. If $m = 0$, p does not matter and the airfoil is symmetric, therefore camber line $\rightarrow y = 0$
 - e. Discretize chord: $x = \text{linspace}(0, 1, 1000)$
 - f. Compute camber height y using NACA piecewise formula:
if $x < p$: $y = (m/p^2)(2px - x^2)$
if $x \geq p$: $y = (m/(1-p)^2)(1 - 2p + 2px - x^2)$
3. If Custom:
 - a. User will input the 3 custom airfoils in the user input file
 - b. Discretize chord: $x = \text{linspace}(0, 1, 1000)$
 - c. Evaluate $y = f(x)$ at all 1000 points
4. Return callable camber function for use in other modules
5. Define plot function and slope function for the camber line. This function takes the returnable function from step 4, along with the title for the plot. We used 1000 points as well for plotting the Camber line.



Algorithm for plotting camber line slope

1. Make a Tuple of Camber function (func) and title (label), and input it to the function
2. Compute the values of this function at discretized values of x (n = 1000)
3. Compute slope using the difference between successive points.
 $z = \text{func}(x)$
 $\text{slope}[i] = (z[i] - z[i-1]) / (1/(n-1))$ {since there n-1 points excluding zero}
4. Since we cannot calculate slope[0] using this method, we assume $z[0] = z[1]$. to ensure continuity.
5. We then plot slope vs x using matplotlib functions.
6. For calculation of slope at a particular point, we assume that slope follows a linear relationship between $z[i]$ and $z[i+1]$ (linear interpolation).
7. We calculate the required m (slope of slope vs x curve) and y intercept and find slope at a point.



Algorithm for computing Fourier coefficients and C_l and C_m

1. Accept camber function $func$, angle of attack aoa_deg , and discretization points n as inputs.

2. Apply coordinate transformation
 $x = 0.5 * (1 - \cos(\theta))$ by discretizing θ from 0 to π into n points, giving corresponding x positions along chord.

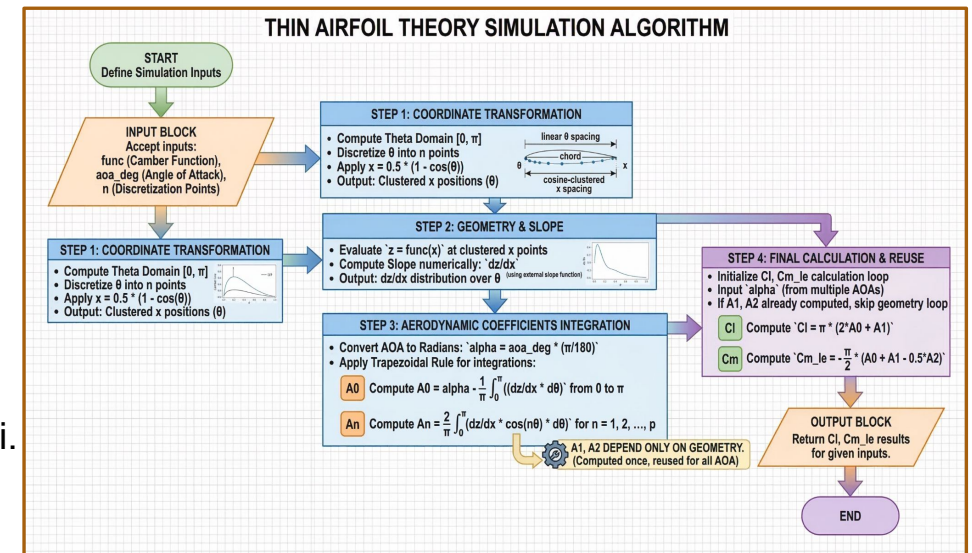
3. Compute slope dz/dx using the function for calculating slope defined earlier

4. Compute A_0 by numerically integrating slope over θ using the trapezoidal rule: $A_0 = \alpha - (1/\pi) * \int_0^\pi (dz/dx * d\theta)$ from 0 to π .

5. Compute A_n ($n = 1, 2, \dots, p$) by multiplying slope by $\cos(n\theta)$ and integrating: $A_n = (2/\pi) * \int_0^\pi (dz/dx * \cos(n\theta) * d\theta)$ from 0 to π .
 Note that A_1 and A_2 depend only on camber geometry, not angle of attack — so they are computed once and reused for all angles.

6. Compute lift coefficient: $C_l = \pi * (2 * A_0 + A_1)$

7. Compute pitching moment coefficient about leading edge: $C_m = -(\pi/2) * (A_0 + A_1 - 0.5 * A_2)$

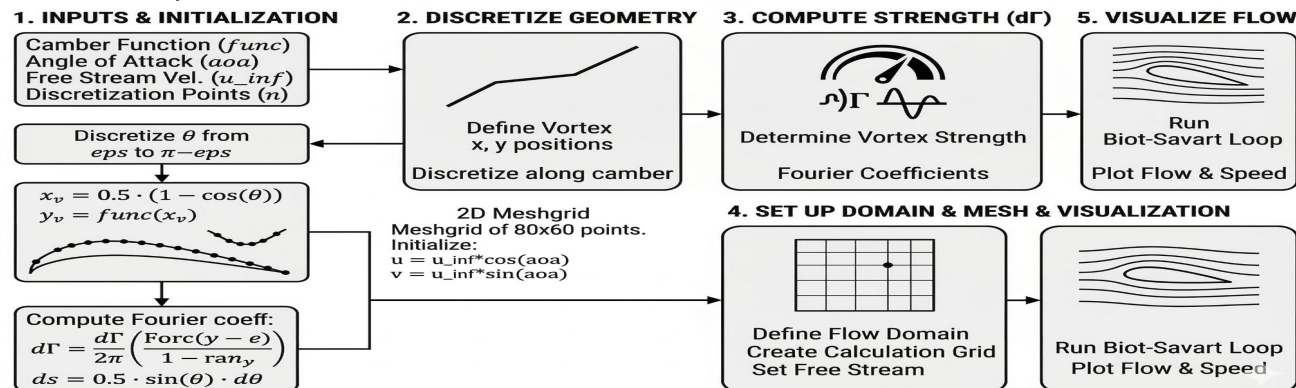


Algorithm for plotting vector field

1. Accept camber function (*func*), angle of attack (*aoa*), free stream velocity (*u_inf*), and discretization points *n* as inputs.
2. Build vortex sheet by discretizing theta from *eps* to $\pi - \text{eps}$ into *n* points. Compute vortex x-positions as $x_v = 0.5 * (1 - \cos(\theta))$ and y-positions as $y_v = \text{func}(x_v)$. Compute circulation distribution $\gamma(\theta)$ using Fourier coefficients. Multiply by arc length element $ds = 0.5 * \sin(\theta) * d\theta$ to get $d\Gamma$ for each element.
3. Define the flow domain as $4c$ along *x* (-1.5 to 2.5) and $3c$ along *y* (-1.5 to 1.5). Create a 2D meshgrid of 80×60 points.
4. Initialise velocity at every grid point with free stream components: $u = u_inf * \cos(\text{aoa})$, $v = u_inf * \sin(\text{aoa})$.
5. For each vortex element at position (*x_v*, *y_v*) with strength $d\Gamma$, compute induced velocity at every grid point using Biot-Savart law:

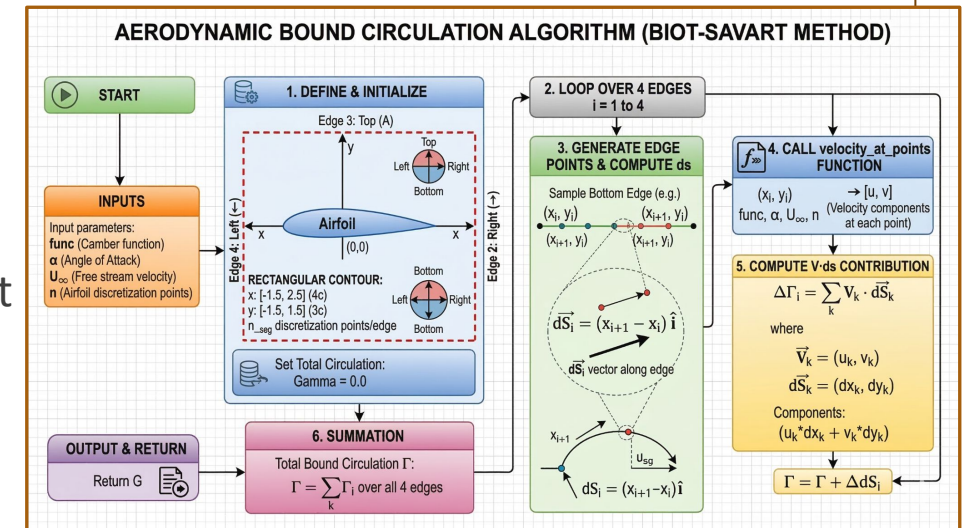
$$u_{\text{induced}} = d\Gamma / (2 * \pi) * (y - y_v) / r^2$$

$$v_{\text{induced}} = -d\Gamma / (2 * \pi) * (x - x_v) / r^2$$
 where $r^2 = (x - x_v)^2 + (y - y_v)^2$, clipped to minimum $1e-8$ to avoid singularity. Accumulate contributions from all vortex elements.
6. Compute speed magnitude at each point: $\text{speed} = \sqrt{u^2 + v^2}$ to plot and assign colour to the contour.
7. Plot filled colour contour of speed magnitude (here I used blue = low and red = high) using `contourf`. Overlay streamlines using `streamplot` to show flow direction. Plot camber line on top for reference.



Algorithm for calculating circulation (through line integral)

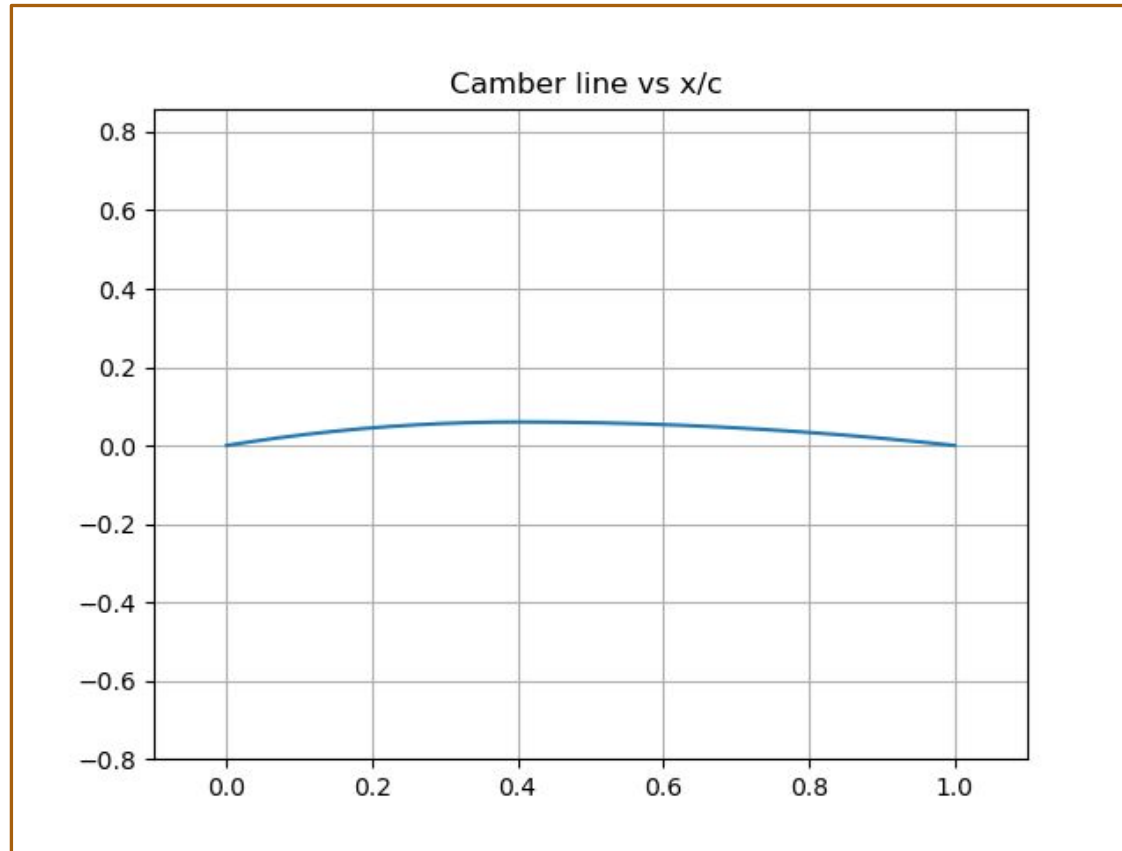
1. Accept camber function func , angle of attack aoa , free stream velocity u_{∞} , and discretization points n as inputs.
2. Define a rectangular contour enclosing the airfoil with boundaries: x from -1.5 to 2.5 ($4c$ domain) and y from -1.5 to 1.5 ($3c$ domain). Discretize each edge into n_{seg} points. Initialise total circulation $\Gamma = 0$.
3. For each of the 4 edges of the rectangle, compute the velocity at every point along that edge using the Biot-Savart law ($\text{velocity_at_points}$ function). This gives velocity components u and v at each point.
4. Compute the contribution to circulation from each edge using V_{ds}
5. Sum all four contributions to get total bound circulation $\Gamma = \oint V \cdot ds$ around the closed contour. Return Γ .



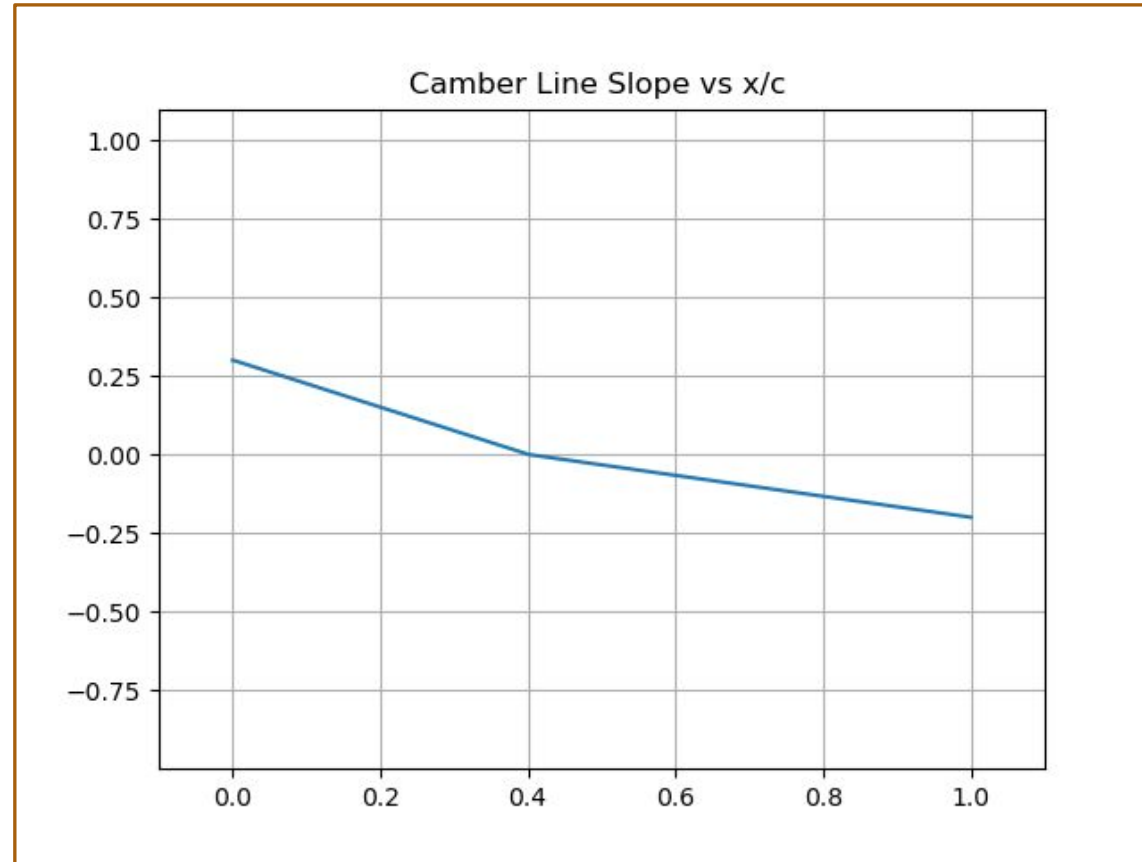
Section 2:

Airfoil Simulation and Results

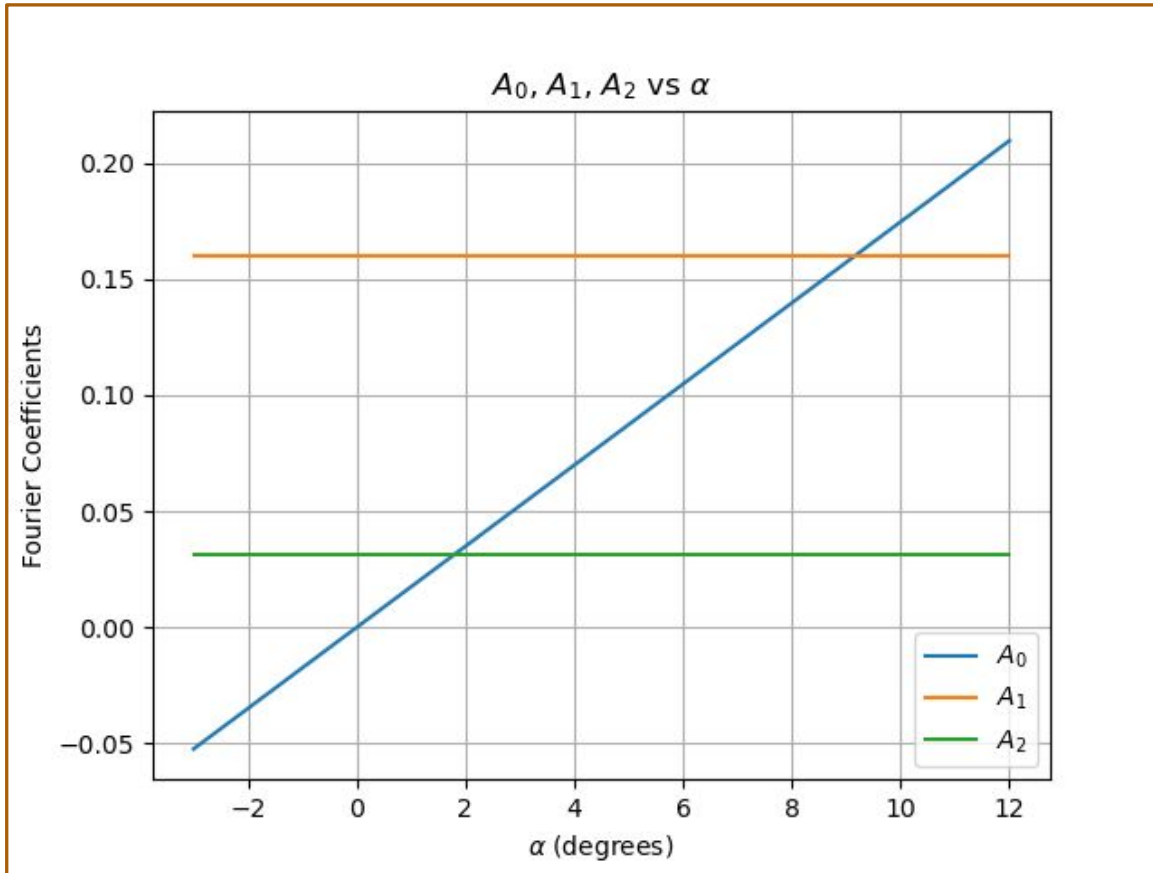
Camber line ('y' vs 'x') as described in Tasks



Camber line slope ('dy/dx' vs 'x')



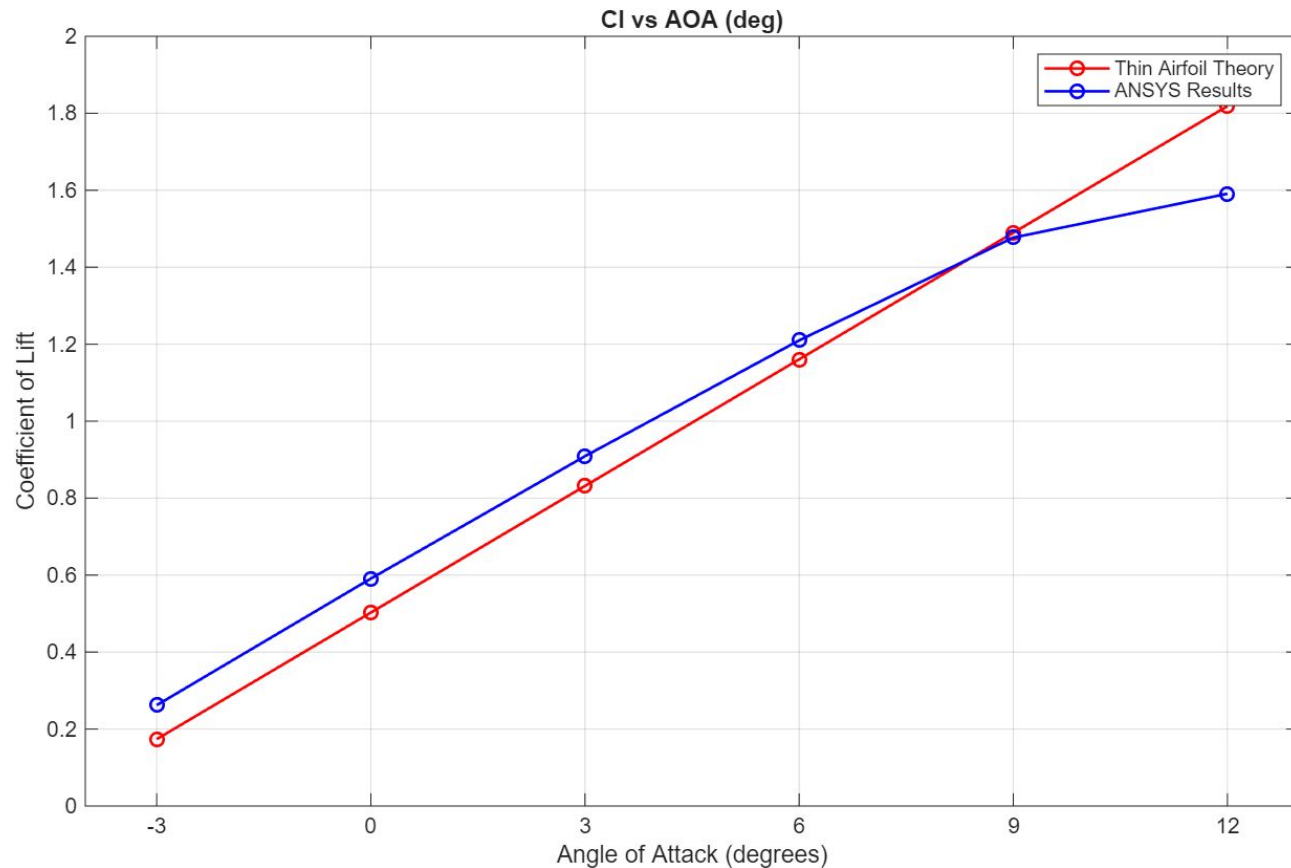
$[A_0, A_1, A_2]$ vs α for the NACA airfoil



3.4: Discussion / Interpretation

- A_0 increases linearly with Angle of attack whereas other coefficients (A_1, A_2) remain constant
- A_1 and A_2 only depend on the gradient and theta at which they are located
- A_0 decreases as we increase camber (dz/dx increases) whereas A_1 and A_2 increase as we increase camber.

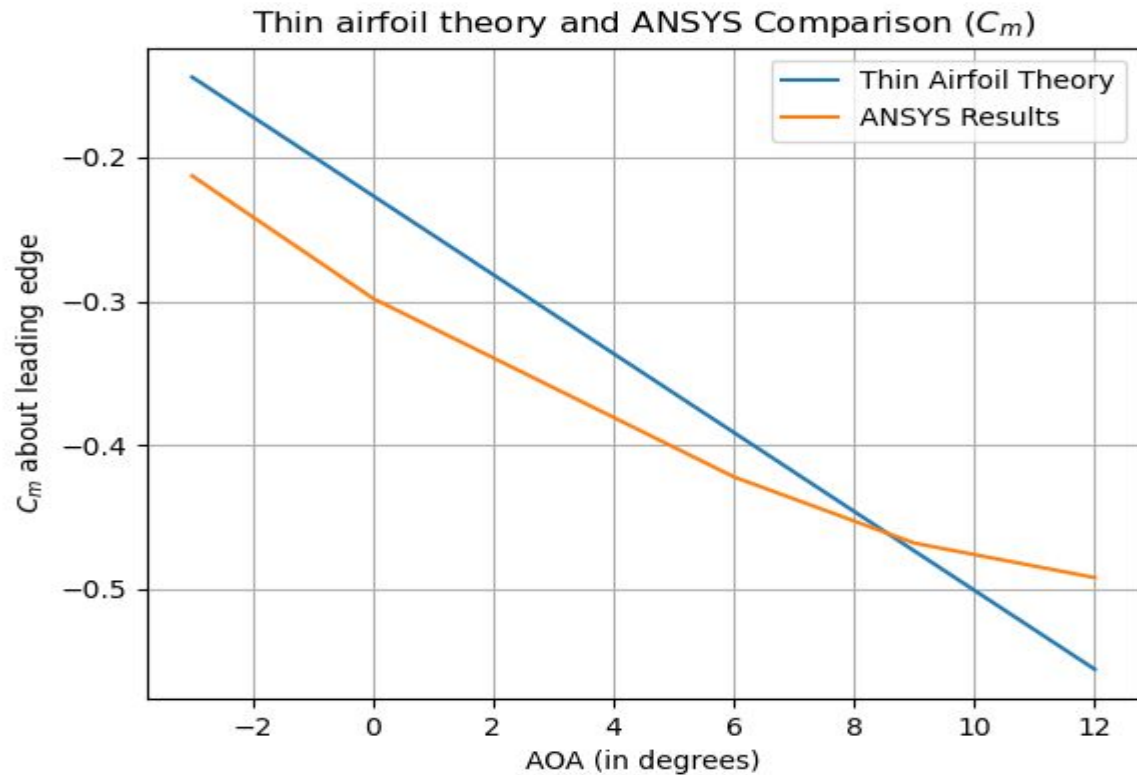
C_l vs α for the NACA airfoil



3.4: Discussion / Interpretation

- Thin airfoil theory does not account for Viscous Effects, hence no boundary layer, hence no separation. Therefore red curve does not saturate (unlike blue curve)
- At low angle of attacks both the curves are nearly parallel with slight offset. I think this is because Adverse pressure gradient is not large enough to force separation. So we only see the effects of viscosity on drag and not lift (for low AOA).
- C_l predicted by thin airfoil theory is lesser at low angle of attacks. My intuition says that since upper surface of the airfoil (in reality) has more curvature/camber than the mean camber line, thus airflow accelerates more over the upper region, therefore creating more suction, Thus More lift

C_m vs α for NACA airfoil

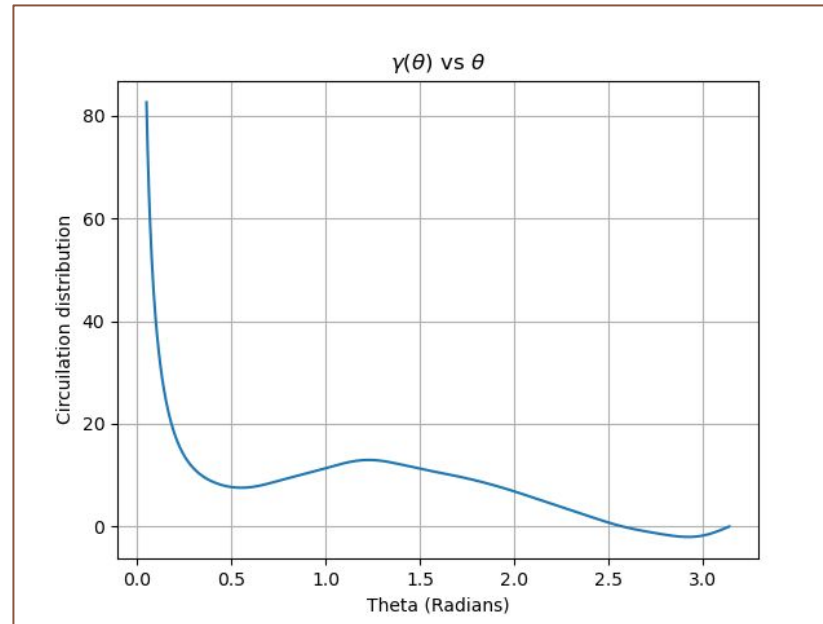
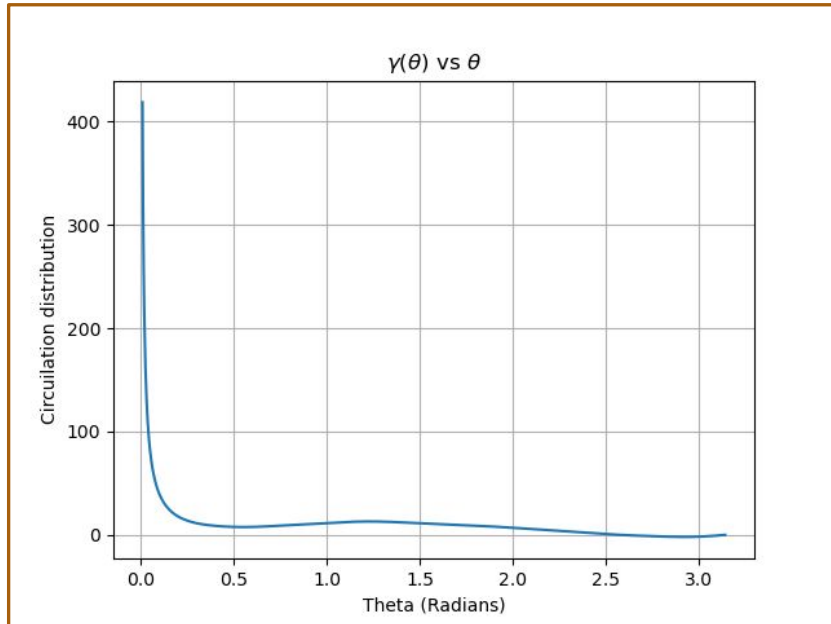


3.6: Discussion / Interpretation

- Coefficient of moment about leading edge is negative, which is a prevalent feature for asymmetric airfoils
- Negative C_m (about leading edge) at zero AOA means that the airfoil has tendency to pitch down even when no lift is being produced. To account for this elevators (or horizontal) tail are made such that they provide negative lift at 0 AOA (for airfoil), to pitch up or cancel out this negative pitch down moment
- We can see deviation in C_m trends (between blue and orange curve) is more when compared to C_l trends
- I think this is because In calculation of C_m in real scenarios drag also plays a role (as drag may not pass through chord line). Also separation happens near trailing edge, since distance to this trailing edge is larger from the leading edge, small deviation in lift produced may have significant effect on C_m values.

Note: In my assignment 1, my C_m vs AOA curve was coming out to be positive with positive slope, I think it was some ANSYS bug, so here I plotted negative of what I originally obtained

Circulation Distribution $\gamma(\theta)$ for the airfoil at $\alpha = 3^\circ$



- I plotted the first graph from 0.01 radians (to avoid division by zero as $\gamma(\theta)$ contains $\sin(\theta)$ in the denominator). But due to very high values of $\gamma(\theta)$ at low θ , we cannot properly see the behaviour at higher θ , therefore I plotted another graph, but this time from 0.05 radians.
- We can see that there is local maxima for $\gamma(\theta)$ near 1.2 radians, which corresponds to the point approximately at $0.4c$ (point of maximum camber).
- Kutta condition is satisfied as $\gamma(c)$ or $\gamma(3.14 \text{ radians}) = 0$.
- Just before trailing edge, we can see γ becomes negative, I think it's because airflow has to slow down and match the airflow's speed below camber line (to satisfy Kutta condition)

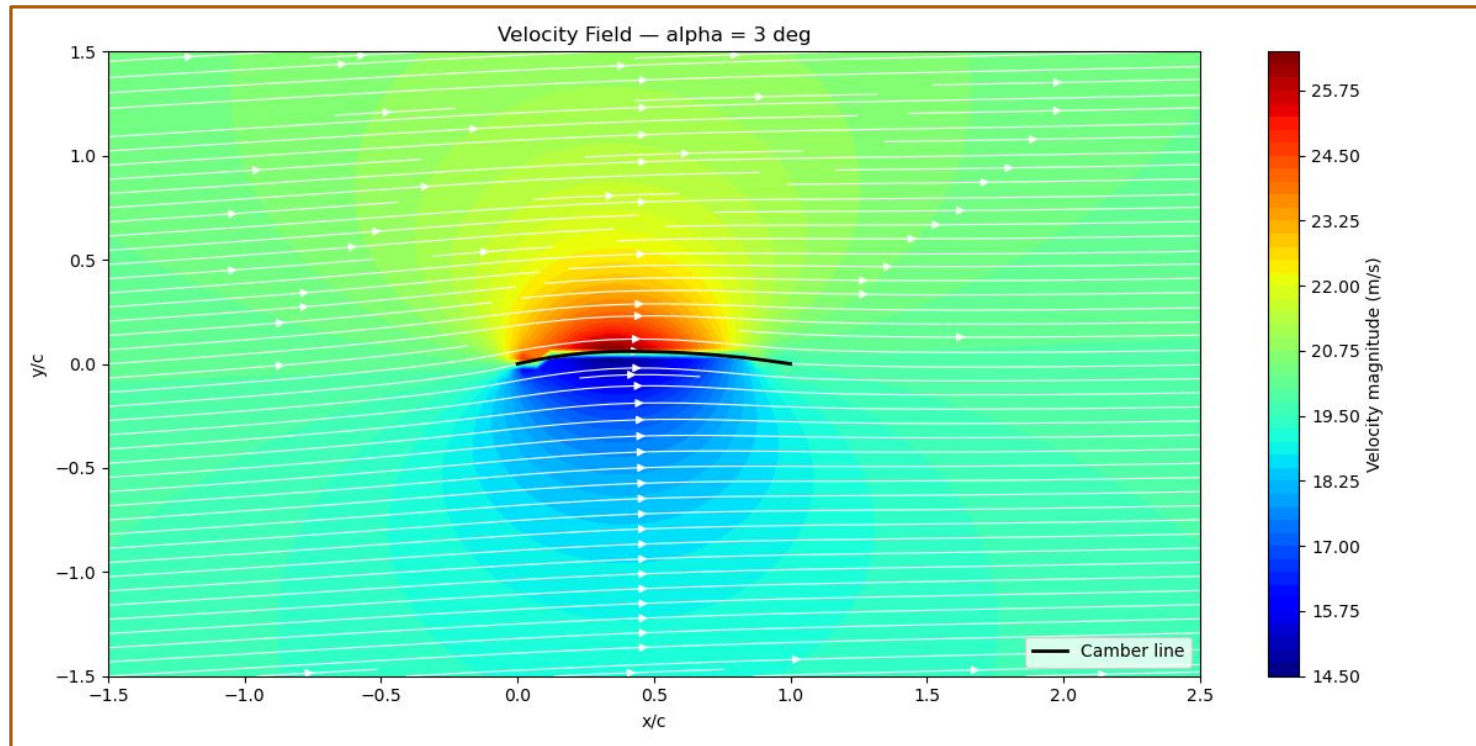
Bound circulation at $\alpha = 3^\circ$

3.8. By integrating circulation distribution	3.11. Velocity line-integral approach
8.275	8.304

3.12: Discussion / Interpretation

- Integration of circulation distribution approach is slightly unreliable (in numerical methods) since we would have to neglect certain interval (near leading edge) since distribution spikes to infinity there.
- Also in my case for summation $\sum_n \sin(n\theta)$, I only took 15 terms, which may also have underestimated the true circulation value.
- Since induced Velocity far from the vortices (near the domain boundaries) are not that heavily dependent on the circulation value, we can infer that velocity line approach is slightly better in terms of accuracy than integrating circulation distribution

Vector field plot around the airfoil (domain size: $4c$ times $3c$) at $\alpha = 3^\circ$



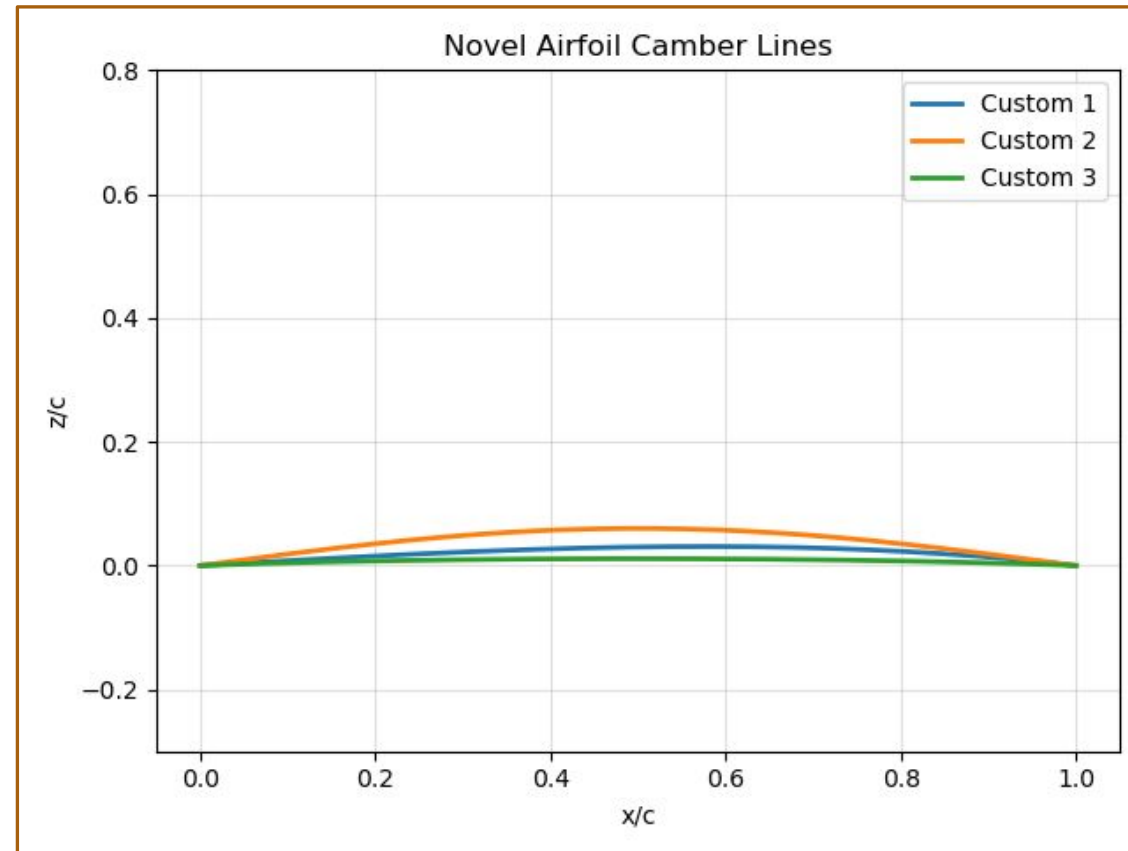
3.10: Discussion / Interpretation

- As expected, airflow is accelerating over the airfoil and slowing down below the airfoil. Creating pressure difference and thus generating lift
- When compared with ANSYS results, Maximum velocity obtained on ANSYS was slightly greater than what is predicted here, which explains lower C_l
- Here we cant figure out stagnation point as there is no leading edge (thin airfoil), therefore flow does not hit any solid wall.
- There is no wake here, as expected due to lack of viscosity
- One difference that I observed when I also checked the ANSYS velocity contour is that higher velocity region is much larger than lower velocity region, which is opposite of what we get here. I think this is because, due to less camber, vortices generated on lower part would be less intense than upper part due to less camber. But here we took sort of average vortices on the mean camber line.

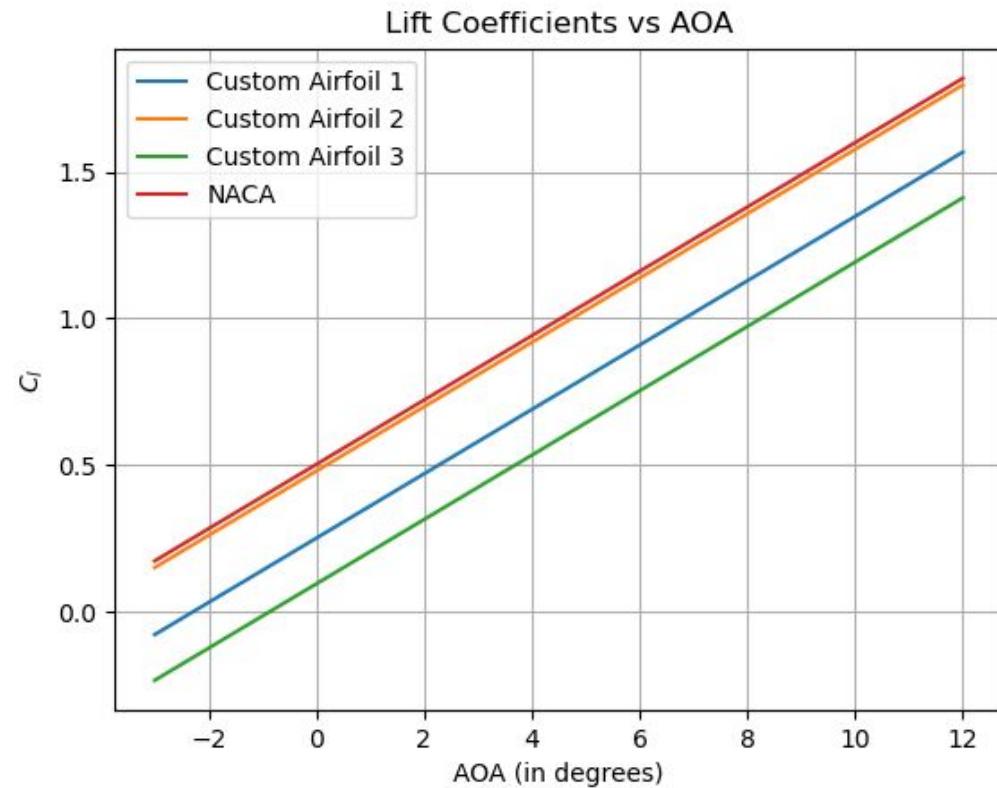
Section 3:

Novel Airfoil Properties

Camber line ('y' vs 'x') for your 3 airfoils



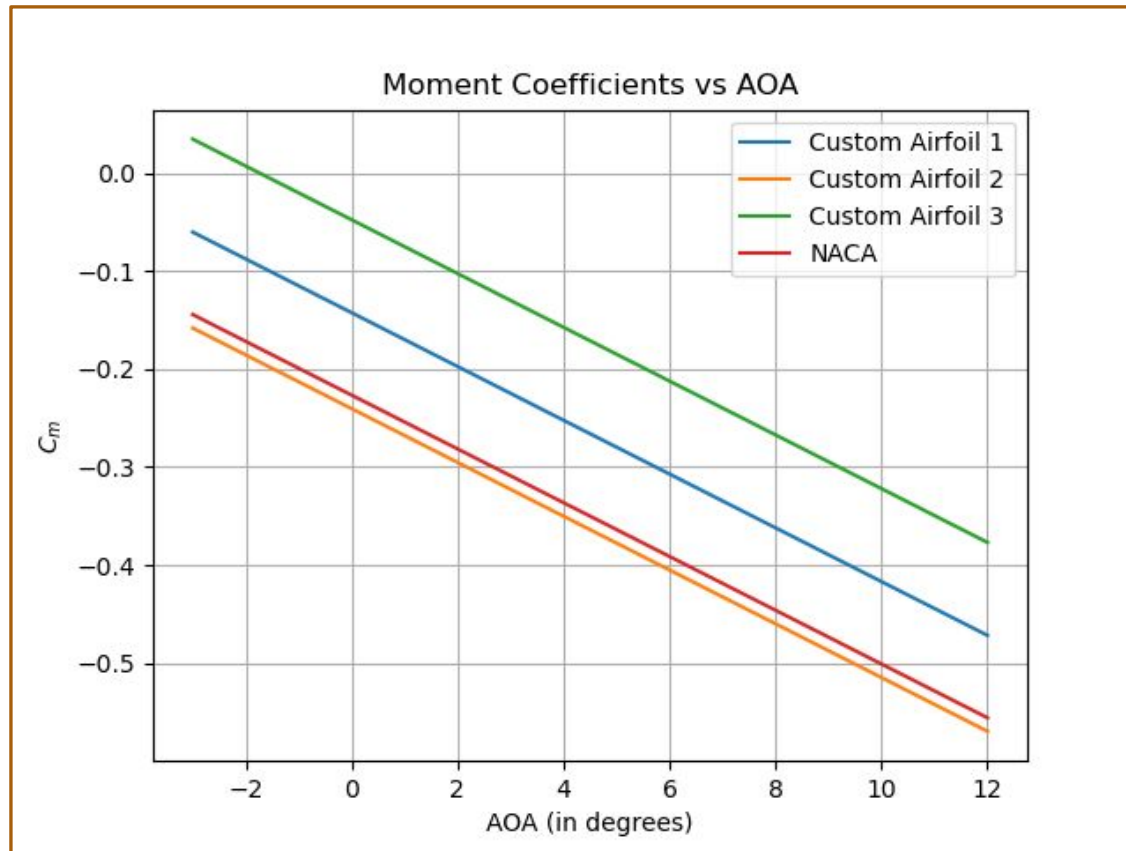
C_l vs α and C_m vs α for your 3 airfoils, along with NACA airfoil



4.3: Discussion / Interpretation

- Since we are using thin airfoil theory, no matter what airfoil we put in, we will always get the same slope for the C_l vs AOA curve ($2\pi\alpha$). Which is not the case with actual airfoils (In assignment 1, customized and original NACA 6412 had different slopes)
 - Out of all the airfoils, Custom airfoil 3 has the least camber, and thus its C_l at zero AOA is almost 0 (approaches the behaviour of symmetric airfoil)
 - They have offsets in them due to the different geometry (mainly the gradient of the function at a particular theta)
- oAccording to this theory, C_l is independent of the flow speed, but in reality (CFD simulations as well), changing Reynolds number may also change C_l

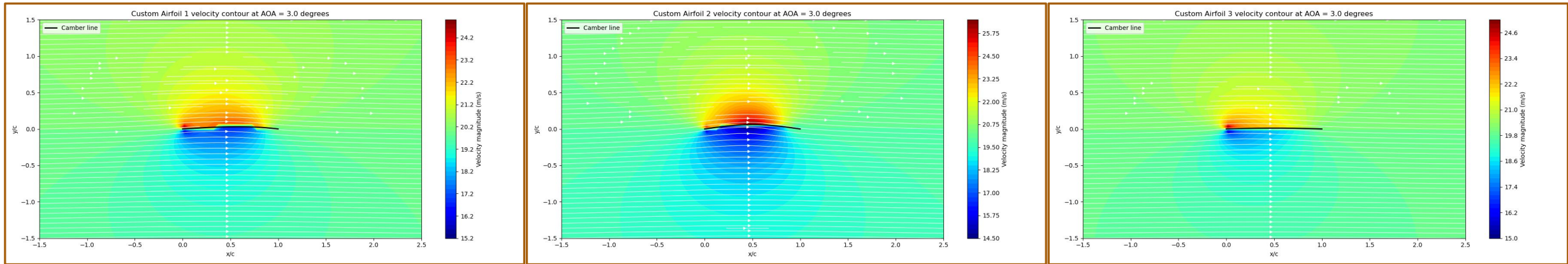
C_l vs α and C_m vs α for your 3 airfoils, along with NACA airfoil



4.3: Discussion / Interpretation

- All these airfoils are asymmetric with positive camber, and thus they all have negative C_m at zero Angles of attack
- Custom airfoil 2 has the most negative C_m values, meaning we would need to have a stronger tail reaction for trimming/balancing
- All these airfoils are statically stable against disturbances which causes change in pitch (because let's say an upward wind gust increases AOA, since at higher AOA C_m value is more negative, airplane will naturally pitch down, correcting for the disturbance)
- From these trends, we can infer that Elevators need to do a lot of work (during Takeoff or landing) to make sure that high AOA of the plane is maintained
- For negatively cambered airfoils, C_m would be positive. Since dz/dx would be negative for such airfoils, thus flipping the sign

Vector field plots around the 3 airfoils



4.5: Discussion / Interpretation

- We can observe that increasing the camber increases the magnitude of induced velocity.
- As discussed earlier, we don't observe any wake.
- I observed that in this code's case, pressure rise below the airfoil is much more than that observed in ANSYS (can be roughly approximated through velocity contour), whereas suction side is generating less suction.

Section 5:

Conclusion

Comparison with Ansys

Overall take on your code's performance as compared to Ansys simulation, and possible reasons for deviations, if any

- I think code performance was good. When we weigh in the efforts vs results, performance of good is justified. We can analyse several airfoils at one, whereas in ANSYS, we would need to make geometries, domain and mesh for each of the Airfoil, which is extremely time taking
- Having said that it's always better to use CFD on shortlisted airfoils. This is because having larger domain captures physics much more accurately. Also Viscous effects in this code are completely ignored, which is not a good approximate, especially if you want to know takeoff, landing and stall characteristics.
- Code would have been much accurate if instead of XX12 airfoil (in my case 6412) airfoil would have been lets say XX06 or something like that, since then assumption that airfoil is thin will be true to some extent.
- Using this method, there is no way of approximating drag, which is essential if we want to know whether our engine is powerful enough to actually takeoff.
- I think possible reason for deviations are mainly (I have already mentioned in discussion part for each plot):
 1. Assuming no viscosity
 2. Thin airfoil assumption (That all vortices lie on mean camber line)
 3. Small angle approximation (which may not hold true for high angles of attack)
 4. Larger domain size in ANSYS and usage of much more accurate models.

Comparison with NACA airfoil

Overall take on the performance of your airfoils as compared to the NACA airfoil

- To be honest I just types random functions that I could come up with (that were 0 on both leading and trailing edges and did not grow rapidly). So Like I am not concerned about their performance
- Coincidentally my custom airfoil 2 is extremely similar to NACA 6412, and that function is nothing but a simple sin function
- Overall, They all underperform when compared to NACA 6412 in terms of lift coefficient.
- In terms of Moment coefficient, there is a trade-off between static stability and elevator effort, which is completely a design choice (more negative C_m more static stability as restoring torque would be greater, but to cancel that restoring torque for level flight, elevators and horizontal tail would have to work harder)